

A Contract-Centered Architecture for Scalable and Manageable Agentic Runtimes

Working preprint | Agentic Runtime Research Project | 2026-07-19

Abstract

Agentic systems are increasingly asked to absorb new capabilities while also serving more concurrent work. These demands are often treated as one scaling problem, although they stress different parts of a runtime. Adding capabilities changes the set of admissible actions and the semantic surface that must be governed; adding capacity changes placement, isolation, scheduling, and resource contention. This paper develops a contract-centered reference architecture that separates those axes. We model an agentic runtime as $A = \langle S, H, X \rangle$: task Skills specify what may be done, a Harness compiles probabilistic intent into governed execution, and Scaffold instances provide isolated physical capacity. The separation is conditional rather than axiomatic. It holds only when Skills and Scaffolds meet through explicit, typed Harness contracts; policy gates are deterministic; activated Skill paths are checked compositionally; and every binding is traceable and replayable. We use this model to derive concrete designs for external-data access, locality-aware parallel execution, planning-time dry-runs, and a Skill-as-Code lifecycle. Because the architecture has not yet been validated through a controlled implementation, we make no performance claims. Instead, we provide falsifiable protocols for capability pollution, physical scaling, Harness bypass, control-plane leakage, path-level safety, locality, and model-version stability. The intended contribution is a research agenda and an implementable vocabulary for scalable agentic runtimes whose growth remains observable, governable, and reversible.

Introduction

The practical problem in an agentic runtime is not simply how to make a large language model (LLM) call more tools. It is how to let a system gain capabilities and capacity without making either dimension opaque. A runtime may add a database Skill, a code-analysis Skill, and a procurement Skill while serving the same number of requests. It may also add workers, sandboxes, accelerators, or regional pools while exposing exactly the same capabilities. The first operation expands a logical action space; the second expands a physical execution space. Combining them in one undifferentiated agent loop hides the distinction that operations teams most need to see.

Consider a common scaling maneuver: every tool description, schema, policy, and example is loaded into every request so that the model can decide what to use. This appears flexible, but it turns capability registration into prompt growth. Unrelated Skills compete for attention, control metadata crosses into request context, and the effect of adding one capability is difficult to localize. A second maneuver adds more workers behind the same loop. Throughput may rise, yet safety and replay do not improve because the runtime still lacks a contract that explains what was planned, which checks ran, and why a particular worker was authorized. More capacity amplifies an execution model; it does not repair it.

This paper asks a narrower question than general agent intelligence: what runtime boundary permits logical capability scaling and physical throughput scaling to evolve independently while remaining manageable? Our answer is the Harness contract. Skill does not directly depend on Scaffold; Scaffold does not understand task-specific Skill semantics. Their dependencies meet only through typed Harness contracts. This is not an unconditional theorem. It is a design claim whose validity depends on mediation, type closure, deterministic gates, and trace identity.

The architecture is organized around a compact model:

$$A = \langle S, H, X \rangle,$$

where S is a versioned set of Skills, H is the Harness that compiles and governs runs, and X is a pool of Scaffold instances. A Skill declares a capability contract. The Harness selects Skills, constructs an execution graph, applies policy and budget checks, binds graph nodes to Scaffold instances, and records evidence. A Scaffold supplies resources and isolation without interpreting the business meaning of the Skill.

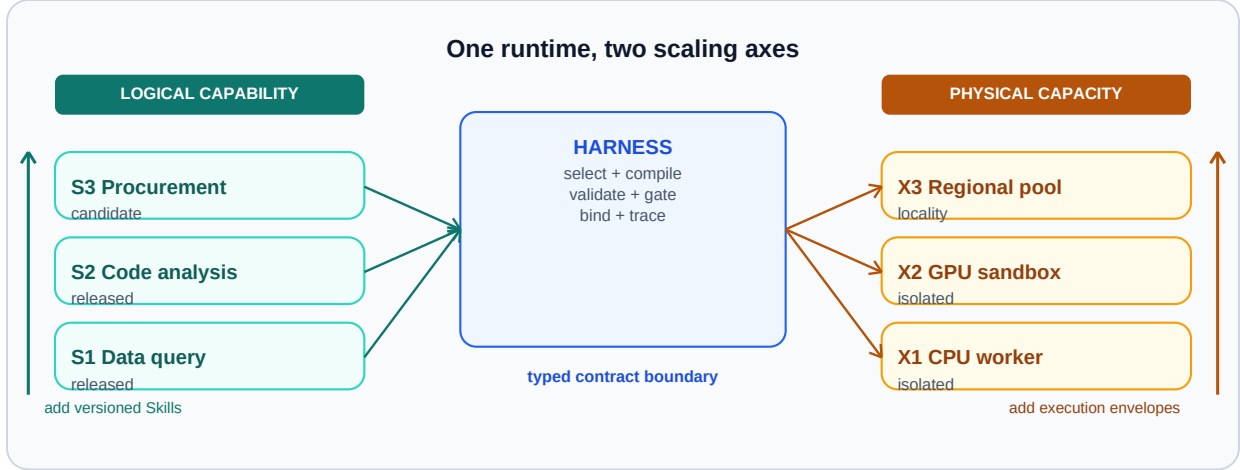


Figure 1. Dual scaling model. Logical extension adds or revises Skills through the Harness contract, while physical extension adds Scaffold capacity through the same binding boundary. Neither axis directly mutates the other. Claim supported: capability growth and throughput growth can be evaluated as distinct interventions when Harness mediation is complete.

Figure 1 establishes the paper's organizing distinction. Horizontal growth changes what the runtime can do; vertical growth changes how much governed work it can execute. The Harness is not merely glue between them. It is the control surface that makes both forms of growth inspectable.

Contributions

This paper makes three primary contributions.

1. We introduce the Skill-Harness-Scaffold model and state the conditions under which logical and physical scaling are separable in an agentic runtime.
2. We define the Harness as a manageable probabilistic control plane: planning may be model-generated, but activation, policy, budgets, binding, evidence obligations, and replay identities are enforced by typed and deterministic mechanisms.
3. We provide falsifiable evaluation protocols rather than synthetic results. The protocols isolate capability pollution, physical scaling, Harness bypass, control-plane leakage, composed-path safety, dry-run economics, and Skill stability across model versions.

The secondary contribution is architectural derivation. External-data access, parallelism and locality, planning-time dry-runs, and Skill-as-Code are not presented as unrelated features. Each follows from treating the Harness contract as the sole semantic-to-physical binding boundary.

Related Work and Novelty Boundary

The closest work is a policy-driven runtime layer for coordinating agent frameworks and serving engines [1]. That line of work identifies an important systems boundary: policies chosen above the serving engine need runtime support below it. Our focus is complementary. We ask how task capabilities themselves are represented, activated, composed, and bound to isolated execution capacity. The distinction matters because scheduling policy can be correct while the capability path remains untyped or unaudited.

Jagarin [2] and Helium [3] explore runtime support for efficient agent execution. Dynamic runtime-graph work [4] similarly moves beyond static workflow templates by constructing graphs at run time. We adopt dynamic graphs inside the Harness, but use Scaffold in a narrower sense: execution and isolation infrastructure such as a sandbox, worker, container, accelerator allocation, or regional pool. It should not be confused with a workflow scaffold or prompt scaffold.

Tool Forge [5] treats tool production and adaptation as a systems problem. Skill-as-Code in this paper shares the asis on lifecycle, but its unit is a validated capability contract rather than only an executable tool. A Skill may include a tool adapter, a planner fragment, schemas, policy tags, tests, and postconditions. Five-plane runtime governance [6] provides a broader governance decomposition; our Harness can be understood as the point at which such governance planes become enforceable on a particular activated path.

Composition safety is especially relevant. Xie et al. show that components benign in isolation may become harmful in composition [7]. This observation directly challenges per-tool allowlists as a sufficient runtime defense. Our response is path-aware checking: the object of authorization is the activated Skill path plus its data and resource bindings, not a bag of individually approved Skills.

Prior agent-memory and retrieval-augmented generation (RAG) systems address context persistence and evidence retrieval [8, 9]. Those mechanisms can be hosted behind a Skill contract, but they do not by themselves establish the capability-to-capacity boundary studied here. The novelty claim is therefore deliberately limited: this paper does not propose a new model architecture, scheduler algorithm, memory representation, or universal governance taxonomy. It proposes a contract boundary and a set of conditions and tests for scalable, manageable agentic runtime behavior.

Model and Assumptions

The Three Runtime Objects

Skill.

A Skill sinS is a versioned declaration of task capability. Its minimum interface is

$$s = \langle g, i, o, p, e, v \rangle,$$

where g describes the goal and applicability conditions, i and o are typed input and output schemas, p contains preconditions and policy tags, e declares externally visible effects, and v identifies the validated version. A Skill can include model instructions, tool adapters, graph fragments, examples, and tests, but these assets are implementation details behind the contract. A Skill is operation-closed when every external effect it can request is declared and testable through its interface.

Harness.

The Harness H is the runtime compiler and governor. Given a request q , identity and policy context u , and a Skill registry S , it produces a candidate execution graph D , validates the graph, binds accepted nodes to Scaffold instances, and emits a trace:

$$(q, u, S) \rightarrow H(D, C, b, \tau).$$

Here C is the resolved Harness contract, b is a binding from graph nodes to Scaffold instances, and τ is the audit and replay trace. Selection and graph construction may use an LLM and are therefore probabilistic. Admission, schema validation, policy evaluation, budget enforcement, effect authorization, and trace creation must be deterministic for a fixed contract and policy snapshot.

Scaffold.

A Scaffold instance xinX is a physical execution envelope:

$$x = \langle r, z, l, a \rangle,$$

where r describes available resources, z describes isolation and trust zone, l describes data and network locality, and a describes attestation and runtime identity. A Scaffold may be a process sandbox, container, virtual machine, browser session, graphics processing unit (GPU) allocation, or remote execution pool. It exposes resource facts and execution primitives, not task semantics.

The Harness Contract

The resolved contract for one run is

```
C=< I, O, G, B, E, T >.
```

I and O are the run-level typed inputs and outputs. G is the activated Skill graph. B contains resource, time, token, cost, and concurrency budgets. E contains authorized effects and evidence obligations. T contains policy, model, registry, data-snapshot, and trace identities needed for replay. This tuple is intentionally operational: each field must be serializable, inspectable, and independently rejectable.



Figure 2. Anatomy of a resolved Harness contract. A probabilistic plan becomes executable only after schemas, graph structure, budgets, effects, evidence obligations, and trace identities have passed deterministic gates. Claim supported: manageability resides in the transition from generated intent to a typed, auditable execution unit.

Figure 2 makes the Harness boundary concrete. A useful shorthand is: probabilistically written, deterministically and audibly executed. The phrase does not imply that execution outcomes are deterministic; external services and models remain variable. It means that the authorization decision, declared effects, selected versions, and evidence requirements can be reconstructed from the recorded contract.

Conditional Decoupling

Logical and physical scaling are conditionally separable under four assumptions.

1. Typed closure. Every activated Skill has typed inputs, outputs, declared effects, and a validated version; undeclared effects fail closed.
2. Complete mediation. A Skill cannot invoke a Scaffold or external system except through a Harness-issued binding.
3. Deterministic gates. Policy, budget, schema, and effect checks produce the same admission decision for the same recorded inputs and versions.
4. Trace identity. The run records enough identity to explain and replay planning, binding, policy, data snapshots, and effects within stated limits.

Under these assumptions, adding s_{new} to S need not change Scaffold semantics, and adding x_{new} to X need not change Skill semantics. The Harness contract is the stable interface. This is a falsifiable engineering claim, not an if-and-only-if theorem: hidden channels, undeclared effects, non-replayable services, or scheduler coupling can break the separation.

Harness Manageability

Control Plane and Data Plane

The control plane owns Skill registration, contract compilation, policy versions, placement rules, and release state. The data plane carries request payloads, executes accepted nodes, moves authorized evidence, and emits traces. The planes interact through references and resolved contracts rather than by copying the full control state into every model prompt.

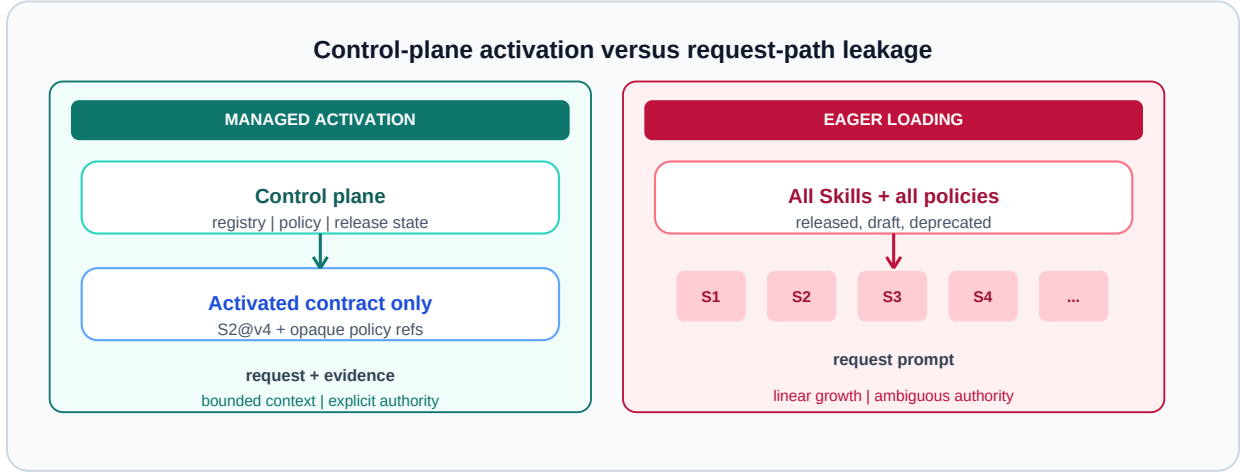


Figure 3. Control-plane separation versus control-state leakage. In the managed design, the request receives only activated contracts and opaque version references. In the leaking design, every Skill schema and policy is injected into the request path. Claim supported: selective activation prevents logical scaling from becoming linear prompt growth and limits exposure of governance metadata.

Figure 3 highlights a failure mode that is easy to mistake for flexibility. Loading every Skill, application programming interface (API) schema, and policy example into each request leaks control-plane state into the data plane. Besides token cost, the leak creates ambiguous authority: a description visible to the planner may look executable even when its release or policy state has changed. The Harness should first select a small candidate set from registry metadata, then materialize only the accepted versions and constraints needed by the run.

Path-Aware Invariants

Per-Skill approval is necessary but insufficient. Let $G_q = (V_q, E_q)$ be the activated graph for request q , with nodes representing Skill operations and edges representing data or control dependencies. The Harness evaluates invariants over the path:

$$\text{Accept}(G_q) = \text{bigwedge}_k \phi_k(G_q, u, B, E, T).$$

Examples include separation of duties, data-residency continuity, prevention of read-then-publish paths, maximum cumulative privilege, and mandatory human approval before an irreversible effect. A path can violate an invariant even when each node is locally allowed. Runtime checks must also cover dynamically inserted nodes and retries, since they change the effective graph.

Audit, Replay, and Reversibility

An audit trace should distinguish three questions: what the planner proposed, what the deterministic gates accepted, and what the bound Scaffold actually executed. Conflating these stages makes incident review nearly impossible. At minimum, the trace records request and principal identity, candidate and activated Skills, contract versions, policy decision inputs, graph hash, binding decisions, external calls, evidence artifacts, and final effects.

Replay has levels. Decision replay recomputes gate outcomes from recorded inputs. Plan replay reruns planning under the recorded model and prompt version, accepting that stochastic output may differ. Execution replay re-executes against pinned data or service snapshots when available. The runtime should declare which level it supports rather than promising exact replay across mutable external systems.

Manageability also requires a stop path. Skill releases can be disabled, policy versions rolled back, bindings revoked, and in-flight graphs cancelled at Harness checkpoints. These mechanisms turn a trace from passive observability into operational control.

Architectural Consequences

External Data as a Contract Instantiation

External data should not be a privileged side channel into the agent. It is a Harness-contract instantiation with explicit authentication, authorization, schema, snapshot, provenance, freshness, residency, and evidence obligations. A data Skill may discover or query assets, but the resolved contract determines what can be fetched and what evidence must accompany the result.

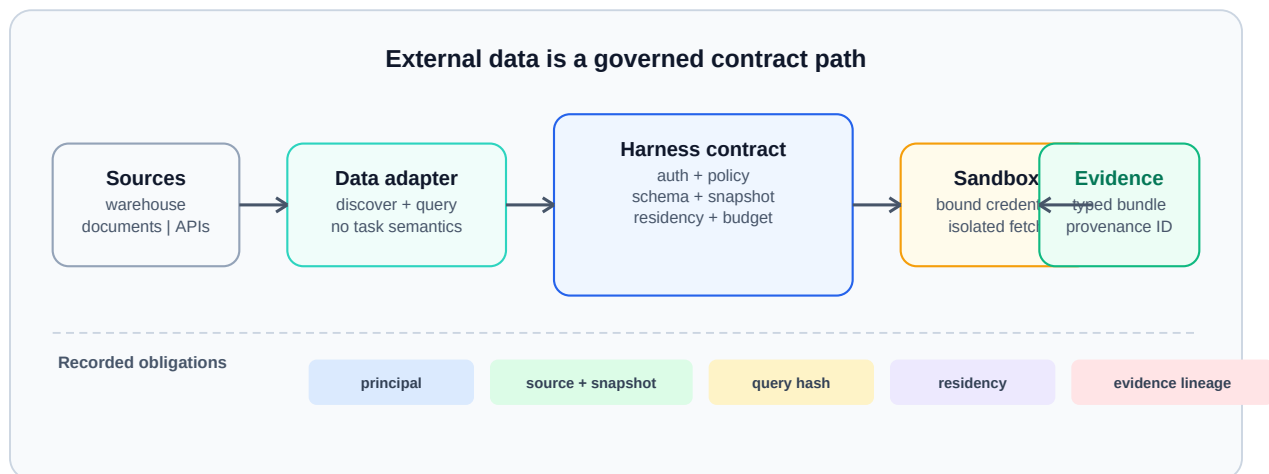


Figure 4. External-data access as a Harness-contract instantiation. Source discovery and query planning remain semantic operations, while credentials, snapshots, provenance, residency, and evidence packaging are resolved and enforced at the contract boundary. Claim supported: data governance can compose with agent execution without embedding infrastructure credentials or source-specific policy in the Skill.

Figure 4 shows how this boundary preserves both portability and accountability. The same analytical Skill can run against different warehouses or document stores when their adapters satisfy the contract. Conversely, the same data source can support multiple Skills without learning their task semantics. Evidence returned to the model is a typed bundle carrying source and snapshot identity, not an unqualified text fragment.

Parallelism, Locality, and Planning-Time Dry-Run

Once the Harness has a graph, it can expose parallelism that a monolithic prompt obscures. Independent nodes may execute concurrently; dependent nodes remain ordered by graph edges and effect constraints. The Scaffold pool contributes locality and capacity facts. The binding algorithm can then co-locate data-intensive nodes, isolate high-risk effects, and reserve scarce accelerators only for nodes that declare them.

A planning-time dry-run resolves graph structure, candidate bindings, budget envelopes, policy decisions, and expected evidence without committing external effects. It answers operational questions before execution: Which data regions will be crossed? Which nodes can run in parallel? Which irreversible effects require approval? Is the predicted cost within budget? Are all required Scaffold types available?

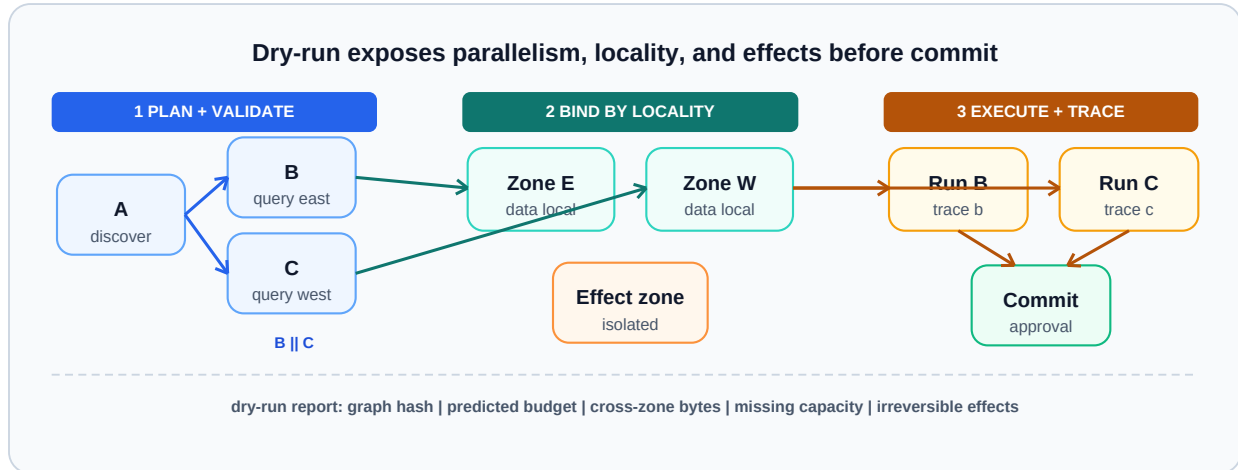


Figure 5. Planning-time dry-run and locality-aware execution. The Harness first validates and annotates the graph, then groups independent nodes near required data and binds effects to appropriate isolation zones. Claim supported: explicit graphs permit concurrency and locality optimization while preserving a reviewable pre-execution decision point.

Figure 5 supports a modest claim: graph visibility creates an optimization opportunity; it does not guarantee a speedup. Dry-run adds planning overhead, and locality can conflict with load balance, policy, or accelerator availability. Evaluation must therefore report both avoided execution cost and dry-run cost.

Skill-as-Code

Treating a Skill as a prompt file makes capability growth difficult to review. Skill-as-Code applies a software lifecycle to the complete capability contract: source-controlled declarations, schema linting, effect review, contract tests, sandbox tests, model-version tests, signed release artifacts, staged rollout, monitoring, and rollback.

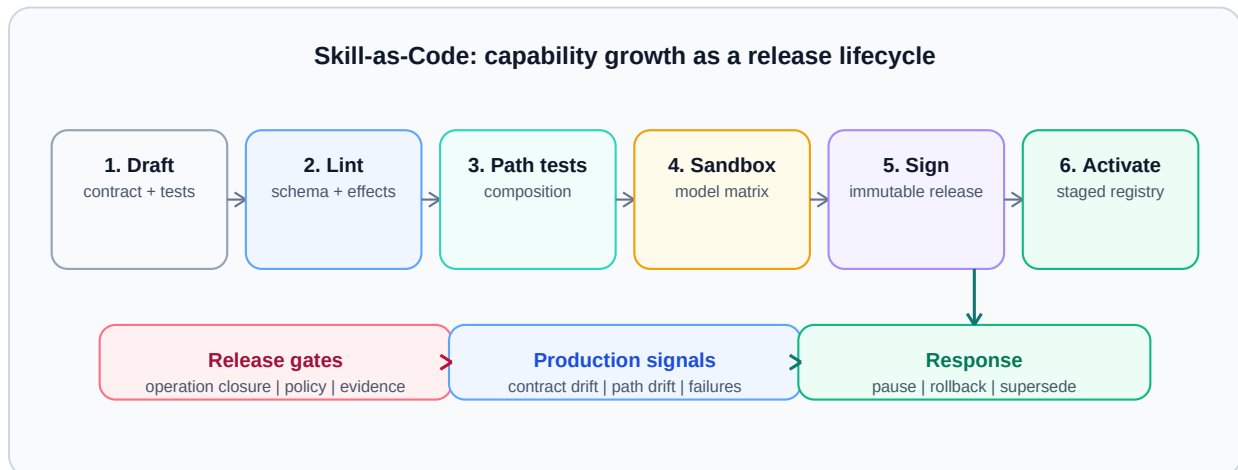


Figure 6. Skill-as-Code lifecycle. A Skill becomes activatable only after contract linting, path and effect tests, sandbox execution, and a signed release; production traces feed monitoring and rollback. Claim supported: logical scaling can be governed as a release process rather than as uncontrolled prompt accumulation.

Figure 6 makes validation part of the architecture rather than a documentation convention. Operation closure is a release gate: tests must show that externally visible behavior is expressible through declared effects. Model-version stability is also tested at this boundary. A Skill can remain logically unchanged while its planner model changes, but the

release should be blocked if contract adherence, path selection, or postcondition satisfaction regresses.

Falsifiable Evaluation Protocols

The present work is architectural and has no controlled implementation results. We therefore specify experiments that can refute its claims. Each protocol separates the intervention from workload, model, policy, and hardware changes. Reported outcomes should include confidence intervals and failure distributions, not only averages.

Falsification matrix			
Claim	Controlled intervention	Primary observations	Counts against claim
Logical scaling	Add unrelated Skills; freeze model and workload	context, selection error, task quality	interference grows with registry
Physical scaling	Add Scaffold instances; freeze Skills and policy	throughput, tail latency, decision consistency	semantics or gates change
Harness mediation	Introduce narrow bypasses	effects, evidence, replay	bypass has no control impact
Path safety	Compose locally allowed Skills	hazard recall, false positives	real hazards remain inexpressible
Dry-run + locality	Toggle dry-run and placement policy	overhead, avoided work, bytes moved	planning costs more than it saves
Skill-as-Code	Change model; freeze Skill source	contract and path stability	material drift passes tests

Figure 7. Evaluation matrix for the architecture. Each row pairs one architectural claim with a controlled intervention, observable metrics, and a condition that would count against the claim. Claim supported: the architecture can be studied through separable, falsifiable experiments rather than through an omnibus benchmark score.

Figure 7 summarizes the research program. The detailed protocols follow.

Logical Scaling and Capability Pollution

Hypothesis. Selective Skill activation through the Harness keeps request-context growth and task interference bounded as unrelated Skills are added.

Control. Hold workload, model, Scaffold pool, and active task Skills fixed. Compare eager prompt loading of all registered Skills with registry retrieval followed by typed activation. Add unrelated Skills in randomized batches.

Metrics. Measure activated-Skill precision and recall, prompt tokens, plan validity, tool-selection errors, task success, policy-decision latency, and accidental exposure of control metadata.

Expected signature. Under selective activation, registry size grows while activated context and task quality remain approximately stable; eager loading exhibits increasing context and more cross-Skill confusion.

Falsification. The claim is weakened if selective activation loses relevant Skills, if its retrieval overhead dominates, or if task interference grows at the same rate as eager loading.

Physical Scaling at Fixed Capability

Hypothesis. Adding compatible Scaffold instances increases sustainable governed throughput without changing Skill behavior or admission semantics.

Control. Freeze Skill, Harness, model, policy, and workload versions. Vary only the number and topology of Scaffold instances. Include homogeneous and locality-constrained pools.

Metrics. Measure throughput, queue time, tail latency, binding failures, isolation failures, cost per completed run, graph completion rate, and gate-decision consistency.

Expected signature. Queueing delay falls and throughput rises until a shared Harness, data, or external-service bottleneck is reached, while contract decisions remain invariant.

Falsification. The separation claim fails operationally if adding Scaffold capacity changes task semantics, bypasses gates, or yields negligible throughput because hidden centralized coupling dominates.

Harness Bypass Ablation

Hypothesis. Complete Harness mediation reduces unauthorized effects and improves decision replay.

Control. Execute the same workloads with full mediation and with narrowly instrumented bypasses: direct tool invocation, untyped data fetch, unrecorded retry, and direct Scaffold selection.

Metrics. Count undeclared effects, policy violations, evidence omissions, non-replayable decisions, incident localization time, and false rejections.

Expected signature. Bypass variants may reduce local latency but produce more unexplained or policy-inconsistent runs.

Falsification. If bypass does not measurably affect control or replay under adversarial workloads, the claimed value of the Harness boundary is overstated or the test lacks meaningful effects.

Control-Plane Leakage

Hypothesis. Passing only activated contracts prevents sensitive or irrelevant registry state from entering request context.

Control. Seed the registry with canary metadata, deprecated schemas, and policy text that is never required by the workload. Compare eager loading, retrieved activation, and opaque-reference activation.

Metrics. Track canary reproduction, prompt tokens, deprecated-tool selection, policy-version confusion, and plan accuracy.

Falsification. The claim fails if control metadata appears in outputs or influences plans despite opaque activation, or if opacity prevents the planner from satisfying valid requests.

Path-Level Composition Safety

Hypothesis. Graph-level invariant checks catch hazards that per-Skill approval misses.

Control. Construct Skill pairs and longer paths that are benign individually but violate data-flow, privilege, or effect-ordering constraints when composed. Compare local allowlists with path-aware admission.

Metrics. Measure hazardous-path detection, false positives, dynamic-graph coverage, and decision latency.

Falsification. The claim is weakened if invariants cannot express realistic hazards, if dynamic insertion routinely escapes checks, or if false positives make valid composition impractical.

Dry-Run and Locality Economics

Hypothesis. Planning-time dry-run and locality-aware binding reduce wasted work or data movement enough to justify their overhead for complex graphs.

Control. Hold graph and Scaffold pool fixed. Compare immediate execution with dry-run admission, and locality-oblivious with locality-aware binding.

Metrics. Report dry-run latency, avoided failed-run cost, bytes moved across zones, completion latency, resource utilization, and policy rejection stage.

Falsification. The feature is not justified where dry-run predictions are poorly calibrated, planning cost exceeds avoided work, or locality choices reduce utilization enough to increase total cost.

Skill-as-Code Across Model Versions

Hypothesis. Contract and path tests identify model upgrades that alter Skill behavior before production release.

Control. Freeze Skill source and test corpus while varying planner and executor model versions. Include perturbations of input phrasing, tool errors, and partial data.

Metrics. Measure schema adherence, declared-effect coverage, path stability, postcondition satisfaction, calibration of abstention, and test flakiness.

Falsification. The lifecycle is insufficient if production-significant changes pass the suite, or if nondeterminism makes the suite too unstable to gate releases.

Discussion

What the Architecture Does Not Solve

The Harness contract cannot make an unreliable model correct, an external service deterministic, or a malicious operating system trustworthy. It does not eliminate semantic ambiguity in Skill descriptions, and it does not prove that a policy captures an organization's intent. It can only make selected assumptions explicit and enforce the parts reducible to runtime checks.

The model also introduces a control-plane concentration risk. If the Harness compiler, registry, or policy evaluator is unavailable, execution may stop. If any is compromised, typed contracts can become a false assurance. Implementations therefore need replicated control services, signed artifacts, independent attestation, and a clearly scoped degraded mode. These are engineering requirements, not consequences guaranteed by the abstract model.

Trade-Offs

Selective activation can miss a relevant Skill. Strict operation closure can reject useful exploratory behavior. Path checking can be computationally expensive for dynamic graphs. Trace retention may conflict with privacy and data minimization. Replay may be impossible when external systems are mutable. Physical decoupling may be limited when a Skill requires specialized hardware or data residency. The architecture makes these couplings visible; it does not wish them away.

Research Questions

Several questions remain open. What is the smallest contract language that captures effects without becoming a second programming language? Which graph invariants can be checked statically, and which require runtime monitors? How should uncertainty from model-generated planning be represented in a deterministic admission decision? How can

traces preserve enough evidence for audit while minimizing sensitive content? At what workload complexity does planning-time dry-run become economical? These questions define the next implementation phase.

Limitations and Threats to Validity

The paper presents a reference architecture, not a production system or empirical result. The model may underrepresent human approvals, long-running state, multi-tenant incentives, and cross-organization trust. The evaluation protocols could favor implementations that expose rich internal telemetry, making comparisons with opaque systems difficult. Synthetic composition hazards may not reflect real incident distributions; real workloads may contain confidential data that cannot be released.

Terminology is another threat. Skill, Harness, and Scaffold are overloaded across agent frameworks. We define them operationally, but mappings to existing systems will be imperfect. In particular, some frameworks combine Harness and Scaffold responsibilities in one service. The model can still be used analytically, but a clean implementation boundary may require refactoring.

Finally, conditional decoupling may hold only within a bounded operating region. A new Skill can require a new hardware class; a new Scaffold can expose timing or locality behavior that changes task quality. Experiments must record these couplings rather than classifying them away. A useful architecture should tell us where separation breaks.

Conclusion

Scalable agentic runtimes have two different growth problems: acquiring more capabilities and executing more governed work. The Skill-Harness-Scaffold model separates them by placing a typed, auditable contract between task semantics and physical execution. The central claim is conditional. Skill and Scaffold can evolve independently only when the Harness completely mediates their interaction, closes declared effects, applies deterministic gates to activated paths, and records identities sufficient for audit and bounded replay.

This framing changes the systems question from "how many tools can the agent see?" to "which capability path was activated, under which contract, on which resources, with which evidence and effects?" It also turns external data, locality, dry-run, and Skill lifecycle into consequences of one boundary rather than a list of platform features. The next step is empirical: implement the boundary, run the falsification protocols, and report where the promised separation holds and where it fails.

References

- [1] Zhang, Kim, Hu. 2026. A Policy-Driven Runtime Layer for Agentic LLM Serving. arXiv preprint arXiv:2605.27744.
- [2] Kadaboina. 2026. Jagarin. arXiv preprint arXiv:2603.05069.
- [3] Wadlom, Shen, Lu. 2026. Helium. arXiv preprint arXiv:2603.16104.
- [4] Xie, others. 2026. From Static Templates to Dynamic Runtime Graphs. arXiv preprint arXiv:2603.22386.
- [5] Rao. 2026. Tool Forge. arXiv preprint arXiv:2605.28000.
- [6] Tallam. 2026. Five-Plane Runtime Governance. arXiv preprint arXiv:2606.12320.
- [7] Xie, others. 2026. Benign in Isolation, Harmful in Composition. arXiv preprint arXiv:2606.15242.
- [8] Packer, Charles, others. 2023. MemGPT: Towards LLMs as Operating Systems. arXiv preprint arXiv:2310.08560.
- [9] Lewis, Patrick, others. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems.